

Processamento da Linguagem Natural

Técnicas de pré-processamento

- Transformar o texto em um formato significativo para análise.
 - **Tokenização:** dividir o texto cru em unidades menores e indivisíveis para processamento.

```
from textblob import TextBlob
import nltk
nltk.download('punkt')
my_text = TextBlob("Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.")
print(my_text.words)
```

Resultado:

```
['Lorem', 'ipsum', 'dolor', 'sit', 'amet', 'consectetur', 'adipiscing', 'elit', 'sed', 'do', 'eiusmod', 'tempor', 'incidunt', 'ut', 'labore', 'et', 'dolore', 'magna', 'aliqua']
```

- **Limpeza dos dados**
 - Remover letras maiúsculas, pontuação, números
 - Remover stop words (palavras com pouco valor semântico)

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
set(stopwords.words('english'))
```

```
"shouldn't", 'so', 'some', 'such', 't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'this', 'those', 'through', 'to', 'too' ...
```

- **Stemming:** reduzir as palavras à sua raiz/base. Ex.: Ferramenta, Ferramentaria, Ferro, Ferragens → Ferr

```
from nltk.stem import PorterStemmer
porter = PorterStemmer()
#Stemmizacao
print("Porter Stemmer")
print(porter.stem("cats"))
print(porter.stem("trouble"))
print(porter.stem("troubling"))
print(porter.stem("troubled"))
```

Similaridade de Documentos

- Count vectorizer e a document-term matrix
 - **Count vectorizer:** Transforma uma coleção de amostras de texto em um vetor de contagens de token.

```
import pandas as pd
corpus = ['Esse é o primeiro documento.', 'Esse é o segundo document
o.', 'E o terceiro. Divirta-se.']
# Count Vectorizer
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
X = cv.fit_transform(corpus).toarray()
pd.DataFrame(X, columns=cv.get_feature_names())
```

	divirta	documento	esse	primeiro	se	segundo	terceiro
0	0	1	1	1	0	0	0
1	0	1	1	0	0	1	0
2	1	0	0	0	1	0	1

- Document-term matrix: descreve a frequência dos termos que ocorrem em uma coleção de documentos
- Bag of words
 - Representação simplificada do texto, na qual cada documento é reconhecido como um conjunto/saco de suas palavras.
 - Estrutura gramatical e a ordem das palavras são desconsideradas, mas mantendo sua multiplicidade.
- Cosine similarity
 - Medida de semelhança entre vetores de um espaço interno do produto que mede o cosseno do ângulo entre eles.

'Eu amo você.'	amo eu pln você					a = [1,1,0,1] b = [1,1,1,0]
	0	1	1	0	1	
	1	1	1	1	0	

$$A \cdot B = \|A\| \|B\| \cos \theta$$

$$\text{Similaridade} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = 0.667$$

- Term frequency-inverse document frequency (TF-IDF)
 - Frequência do termo - frequência inversa de documento, é uma estatística numérica que visa refletir a importância de uma palavra para um documento em uma coleção ou corpus.

$$TFIDF = (\text{Frequência do Termo}) * (\text{Inverso da Frequência nos documentos})$$

Onde:

$$\text{Frequência do Termo} = \frac{\text{Contagem do Termo no Documento}}{\text{Total de Termos no Documento}}$$

$$\text{Inverso da Frequência nos documentos} = \log \left(\frac{\text{Total de Documentos} + 1}{\text{Documentos contendo o termo} + 1} \right)$$

```
import pandas as pd
corpus = ['Esse é o primeiro documento.', 'Esse é o segundo document
o.', 'E o terceiro. Divirta-se.']
# TF-IDF Vectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
cv_tfidf = TfidfVectorizer()
X_tfidf = cv_tfidf.fit_transform(corpus).toarray()
pd.DataFrame(X_tfidf, columns=cv_tfidf.get_feature_names())
```

	divirta	documento	esse	primeiro	se	segundo	terceiro
0	0.00000	0.517856	0.517856	0.680919	0.00000	0.000000	0.00000
1	0.00000	0.517856	0.517856	0.000000	0.00000	0.680919	0.00000
2	0.57735	0.000000	0.000000	0.000000	0.57735	0.000000	0.57735

Aplicação Processamento da Linguagem Natural na base de dados de Análise de Sentimentos de texto Twitter

Sentiment Analysis Tweets Dataset

Descrição: Análise de sentimentos Tweets de seis companhias aéreas. Prever se um tweet contém sentimentos positivos, negativos ou neutros sobre uma companhia aérea.

Filename: Tweets.csv

Colunas:

airline_sentiment
text

Exemplo:

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	airline	airline_sentiment_gold	name
0	570306133677760513	neutral	1.0000	NaN	NaN	Virgin America	NaN	cairdin
1	570301130888122368	positive	0.3486	NaN	0.0000	Virgin America	NaN	jnardino
2	570301083672813571	neutral	0.6837	NaN	NaN	Virgin America	NaN	yvonnalynn
3	570301031407624196	negative	1.0000	Bad Flight	0.7033	Virgin America	NaN	jnardino
4	570300817074462722	negative	1.0000	Can't Tell	1.0000	Virgin America	NaN	jnardino

Aplicação Processamento da Linguagem Natural na base de dados de detecção personagens do South Park

Detecção Personagens South Park Dataset

Descrição: Análise dos diálogos dos personagens do South Park.

Filename: All-seasons.csv

Colunas:

Season

Episode

Character

Line

Exemplo:

	Season	Episode	Character	Line
0	10	1	Stan	You guys, you guys! Chef is going away. \n
1	10	1	Kyle	Going away? For how long?\n
2	10	1	Stan	Forever.\n
3	10	1	Chef	I'm sorry boys.\n
4	10	1	Stan	Chef said he's been bored, so he joining a gro...